المدرسة الوطنية العليا للذكاء الاصطناعي وعلوم المعطيات - تارودانت

ⵜⴰⵙⴷⴰⵡⵉⵜ ⵜⴰⵏⴰⵎⵓⵔⵜ ⵏ ⵓⵙⵏⵉⵙ ⴰⵏⴰⴼⵍⵍⴰ ⴷ ⵜⵎⴰⵙⵙⴰⵏⵉⵏ - ⵜⴰⵔⵓⴷⴰⵏⵜ

ÉCOLE NATIONALE SUPÉRIEURE DE L'INTELLIGENCE ARTIFICIELLE ET SCIENCES DES DONNÉES – TAROUDANT

Université Ibn Zohr - Traoudant

École Nationale Supérieure des Sciences des Données, Big Data et Intelligence Artificielle

Filière : Science de Donne, Big Data et Intelligence Artificielle

ARGAN-FIRE WATCH SYSTÈME DE DÉTECTION DES INCENDIES DES ARGANIERS

Préparé par :

Hajar GHAZAF

Abdelmalek HAMDA

Assmaa HASSAN

Encadré par :

Pr. Sara El-ATEIF

Soutenu le 15 Avril 2025

Année universitaire 2025/2026

REMERCIEMENTS

Nous tenons à remercier Allah pour la force, la santé et la persévérance qu'Il nous a accordées tout au long de ce projet.
Nous exprimons également notre gratitude à notre professeur et encadreur pour ses conseils, son suivi et sa disponibilité qui ont grandement contribué à la réalisation de ce travail.

TABLE DES MATIERES

Remerciements 3
Table des matières 5
Introduction générale 6
CHAPITRE 1 : Introduction et Description du Projet 7
I. Introduction 8
II. Contexte Général 8
1-Problématique 8
2-Besoins Fonctionnels 8
3-Besoins Non Fonctionnels 9
4-Acteurs du système 9
5-Exigences Techniques 9
III-Conclusion 10
CHAPITRE 2 : Analyse et conception 11
I. Introduction 13
1-Rôle de l'analyse et de la conception 13
2- Qu'est-ce qu'un modèle conceptuel ? 13
3- Présentation de la méthode merise 13
4- Présentation du langage UML 14
II. Modèle conceptuel des données 14
III. Modèle logique des données 16
IV. Modèle physique des données 18
V. Diagramme des cas d'utilisation 21
1-Diagramme des cas d'utilisation pour l'administrateur 21
2-Diagramme des cas d'utilisation pour un pompier 23
3-Diagramme des cas d'utilisation pour un responsable d'une coopérative 24
VI. Script SQL : Triggers et procédures 25
1-Procédures 25
2-Triggers 26
VII. Conclusion 29

CHAPITRE 3 :Outils ,technologies et prompts utilisés 30
I. INTRODUCTION 31
II. Technologies Utilisées 31
1-Langages et Frameworks Utilisés 31
a-Python 31
b-Flask 31
c-React 32
d-MySQL 32
2-Outils de Développement 32
a-VS Code 32
b-Antigravity 32
c-Looping 33
3-Outils d'Intelligence Artificielle 33
III. Prompts Utiliés 33
IV. Conclusion 40
Conclusion générale 41

TABLE DES FIGURES

Figure 1 : Logo UML 20
Figure 2 : Modèle conceptuel des données 22
Figure 3 : Modèle logique des données 23
Figure 4 : Diagramme des cas d'utilisations pour l'administrateur 25
Figure 5 : Diagramme des cas d'utilisations pour le pompier 26
Figure 6 : Diagramme des cas d'utilisations pour le responsable de la coopérative 27
Figure 7 : Logo Python 28
Figure 8 : Logo Flask 29
Figure 9 : Logo React 31
Figure 10 : Logo MySQL 32
Figure 11 : Logo VS Code 32
Figure 12 : Logo Antigravity 32
Figure 13 : Logo Looping 33
Figure 14 :Logo ChatGPT 33
Figure 15 : Logo Grok 33
Figure 16 : Logo Claude 33
Figure 17 : Logo DeepSeek 33

INTRODUCTION GENERALE

Dans ce rapport, nous présentons **Argan Fire Watch**, une plateforme de surveillance et d'alerte incendie conçue pour protéger les forêts d'arganier dans la région de Souss-Massa. Face à l'augmentation des risques d'incendies due au changement climatique, à la sécheresse et aux activités humaines, il devient essentiel de disposer d'un outil moderne et efficace pour détecter rapidement les départs de feu et coordonner les interventions.

Notre système vise principalement à offrir une surveillance en temps réel des zones forestières grâce à des capteurs IoT, à générer des alertes précoces et à faciliter la communication entre les coopératives locales, les pompiers et l'administration. En proposant une solution technologique adaptée au contexte marocain, nous souhaitons contribuer à la préservation de cet écosystème unique, classé au patrimoine mondial de l'UNESCO, tout en soutenant les communautés qui dépendent de la forêt d'argan.

Grâce à des fonctionnalités avancées telles que le suivi des capteurs en direct, la cartographie interactive des zones à risque, le système d'alertes via WhatsApp et les tableaux de bord adaptés à chaque rôle (administrateur, coopérative et pompier), Argan Fire Watch apporte une réponse concrète aux défis de la gestion des incendies en milieu forestier.

Le développement d'Argan Fire Watch repose sur une architecture full-stack moderne combinant **Flask (Python)** pour le backend, **React** pour le frontend et **MySQL** pour la base de données. L'interface utilisateur a été conçue pour être intuitive et accessible, permettant à tous les acteurs concernés de consulter les informations et de réagir rapidement en cas d'urgence.

CHAPITRE 1 :
INTRODUCTION ET DESCRIPTION DU PROJET

I. INTRODUCTION

Dans ce chapitre, nous allons présenter notre projet en détaillant son contexte, ses objectifs et les acteurs impliqués. Nous commencerons par analyser la problématique actuelle liée à la gestion des incendies dans les forêts d'arganier, en identifiant les défis rencontrés par les coopératives, les pompiers et les autorités. Ensuite, nous définirons les objectifs à atteindre à travers cette plateforme et mettrons en avant les avantages qu'elle apporte. Enfin, nous présenterons les outils de développement et les choix technologiques adoptés pour la conception et la mise en œuvre de ce système.

II.CONTEXTE GÉNÉRAL

1-Problématique

Les forêts d'arganier de la région Souss-Massa font face à une recrudescence des incendies causée par le changement climatique, la sécheresse prolongée et certaines activités humaines. Actuellement, la surveillance repose principalement sur des méthodes traditionnelles (tours de guet, patrouilles) qui manquent de réactivité et de couverture efficace sur de vastes zones difficiles d'accès. Cela entraîne des délais de détection importants, une propagation rapide des feux et des difficultés de coordination entre les différents acteurs (coopératives locales, services de pompiers et administration). L'absence d'un système de monitoring en temps réel et d'alertes précoces accentue les pertes environnementales, économiques et humaines.

2-Besoins Fonctionnels

La plateforme Argan Fire Watch répond aux besoins suivants :

- Surveillance en temps réel : Suivi continu des zones forestières via des capteurs IoT mesurant la température, l'humidité et la vitesse du vent.
- Détection et alertes précoces : Génération automatique d'alertes avec différents niveaux de gravité et envoi de notifications instantanées via WhatsApp.
- Gestion des zones forestières : Création et visualisation interactive des zones à risque sur une carte avec dessin de polygones.
- Tableaux de bord adaptés : Interfaces spécifiques selon le rôle (Administrateur, Coopérative et Pompier).
- Gestion des interventions : Suivi des alertes, coordination des équipes de pompiers et rapport d'intervention.
- Interaction avec les utilisateurs : Permettre aux coopératives de gérer leurs capteurs et zones, et aux pompiers de consulter les incidents en cours.

3-Besoins Non Fonctionnels

- Performance : Assurer un temps de réponse rapide pour l'affichage des données en temps réel et la génération d'alertes.
- Fiabilité : Garantir la disponibilité continue du système, même en zones rurales à faible connectivité.
- Sécurité : Protéger les données des utilisateurs et des zones sensibles via l'authentification JWT et le chiffrement des communications.
- Évolutivité : Permettre l'ajout futur de fonctionnalités telles que l'intelligence artificielle pour la prédiction de propagation des feux.
- Accessibilité : Interface intuitive et adaptée aux différents profils d'utilisateurs (coopératives et pompiers).

4-Acteurs du système

Le système Argan Fire Watch implique trois principaux acteurs :

- Administrateur : Gère les comptes des coopératives, approuve les inscriptions, supervise l'ensemble du système et consulte les statistiques globales.
- Coopérative : Propriétaire des zones forestières, elle gère ses capteurs, visualise ses zones et reçoit les alertes concernant ses parcelles.
- Pompier / Chef d'équipe : Consulte les alertes en cours, visualise la localisation des incendies sur la carte et gère les interventions sur le terrain.

5-Exigences Techniques

- Développement Full-Stack : Backend développé avec le framework Flask (Python) et frontend avec React.
- Base de données : MySQL pour le stockage des données, avec support des données spatiales pour la gestion des zones forestières.
- Cartographie : Utilisation de Leaflet et React-Leaflet pour l'affichage interactif des cartes et le dessin des polygones.
- Notifications : Intégration de Twilio pour l'envoi d'alertes via WhatsApp.
- Conception des Interfaces : Interfaces utilisateur modernes, responsives et intuitives, adaptées aux différents rôles grâce à Tailwind CSS et Framer Motion.

III-Conclusion

En conclusion, le projet **Argan Fire Watch** répond à une problématique critique et actuelle : la vulnérabilité croissante des forêts d'arganier face aux incendies, exacerbée par le changement climatique et les moyens de surveillance traditionnels inefficaces. Face à ce constat, notre plateforme propose une solution innovante, basée sur des technologies modernes (IoT, cartographie interactive, alertes automatisées) pour assurer une détection précoce, une meilleure coordination entre les acteurs (coopératives, pompiers, administrateurs) et une réactivité accrue.
Les besoins fonctionnels et non fonctionnels identifiés ont guidé nos choix techniques : un backend robuste avec Flask, une base de données MySQL intégrant des données spatiales, un frontend dynamique avec React et Leaflet, ainsi qu'un système de notification via Twilio pour des alertes instantanées sur WhatsApp. L'architecture retenue garantit performance, fiabilité, sécurité et évolutivité, tout en offrant des interfaces adaptées à chaque profil d'utilisateur.
Ainsi, Argan Fire Watch constitue une réponse concrète et durable pour la préservation du patrimoine forestier unique de la région Souss-Massa, en transformant la surveillance passive en un système proactif, collaboratif et intelligent. Ce projet ouvre également la voie à des améliorations futures, comme l'intégration de l'intelligence artificielle pour la prédiction de la propagation des feux, renforçant encore son impact environnemental et socio-économique.

CHAPITRE 2 :
ANALYSE ET CONCEPTION

I. INTRODUCTION

Avant de commencer le développement, il est essentiel d'analyser les besoins et de concevoir une structure claire de l'application. Ce chapitre présente les besoins exprimés à travers le cahier des charges, traduits à l'aide des modèles conceptuels. Nous y exposons les principaux diagrammes réalisés : le modèle conceptuel des données (MCD), le modèle logique des données (MLD) et le diagramme des cas d'utilisation, afin de définir précisément le fonctionnement et l'architecture de l'application.

1-Rôle de l'analyse et de la conception

L'analyse et la conception sont des étapes fondamentales dans tout projet de développement logiciel. Elles permettent de traduire les besoins du client en une vision claire et structurée du système à développer. En séparant la réflexion sur la structure de l'application du codage lui-même, on améliore la compréhension du projet, la détection des erreurs en amont et la communication entre les intervenants.

2- Qu'est-ce qu'un modèle conceptuel ?

Un modèle conceptuel est une représentation abstraite du système, centrée sur la compréhension des besoins fonctionnels sans se soucier des aspects techniques. Il permet de définir les principales entités manipulées, leurs attributs, et les relations entre elles. Ce modèle constitue une base solide pour la conception technique, car il exprime clairement la logique métier et les attentes fonctionnelles.

3- Présentation de la méthode merise

Pour modéliser l'application, nous avons utilisé la méthode **MERISE**, une approche largement adoptée dans le domaine des systèmes d'information. Elle permet de structurer et d'organiser les données ainsi que les traitements de manière claire et cohérente.
MERISE s'appuie sur une démarche progressive basée sur plusieurs niveaux de modélisation (conceptuel, logique et physique), permettant de passer d'une vision abstraite du système à une implémentation concrète dans une base de données.
Dans ce contexte, MERISE nous a permis de définir et d'organiser efficacement les différentes entités du système ainsi que leurs relations.

4- Présentation du langage UML

En complément de la méthode MERISE, nous avons utilisé le langage **UML** afin de modéliser les aspects fonctionnels et dynamiques du système. UML est un langage de modélisation standard qui permet de représenter graphiquement la structure et le comportement d'un système à travers différents types de diagrammes.
Son utilisation facilite la compréhension et la communication autour du projet, en offrant une vision claire des interactions entre les acteurs et des fonctionnalités du système. Ainsi, UML vient compléter MERISE en apportant une représentation visuelle et détaillée du fonctionnement global de l'application.

Figure 1 : Logo UML

II. Modèle conceptuel des données

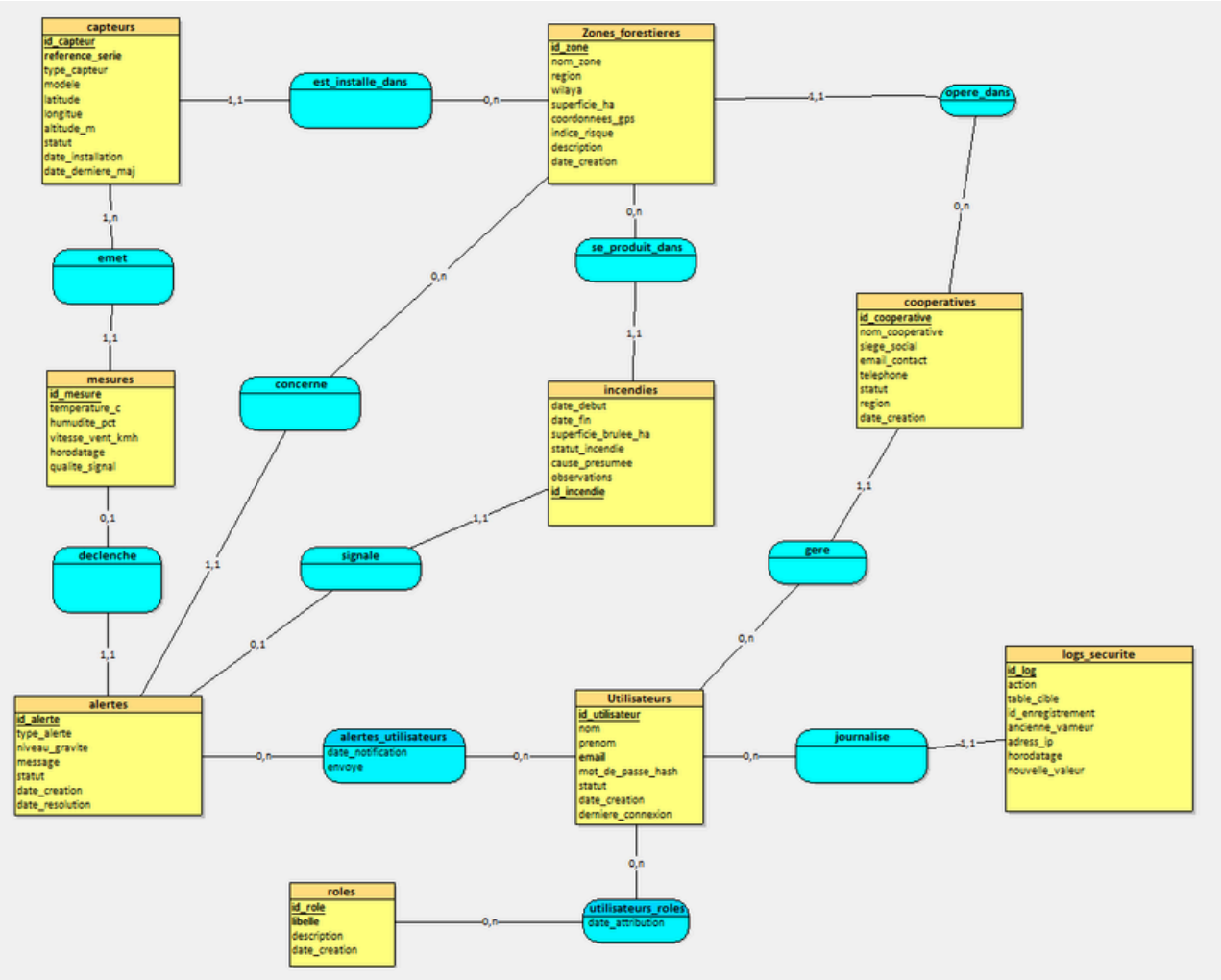

Figure 2 : Modèle conceptuel des données

Le diagramme ci-dessus représente le Modèle Conceptuel de Données (MCD) du système étudié. Il illustre les différentes entités ainsi que les relations qui les lient.

Au centre de ce modèle, l'entité zone forestière représente les zones à surveiller, caractérisées par des informations telles que la localisation, la superficie et le niveau de risque. Des capteurs sont installés dans ces zones et assurent la collecte continue de données environnementales à travers l'entité mesure, incluant notamment la température, l'humidité et la vitesse du vent.

La logique métier du système repose sur l'analyse de ces mesures. En effet, lorsque certaines valeurs dépassent des seuils prédéfinis, une alerte est automatiquement déclenchée. Cette alerte est ensuite transmise aux utilisateurs du système, en particulier ceux ayant un rôle de pompier, afin de permettre une intervention rapide.

Suite à cette intervention, si un incendie est confirmé, un enregistrement est créé dans l'entité incendie, contenant les informations relatives à l'événement, telles que la date de début, la superficie brûlée et le statut. Chaque incendie est associé à une zone forestière spécifique.
Par ailleurs, le système intègre une dimension organisationnelle à travers les coopératives, qui opèrent dans les zones forestières. Les utilisateurs, rattachés à ces coopératives, interagissent avec le système en recevant des alertes et en effectuant diverses actions. Ces actions sont enregistrées dans un journal de sécurité, garantissant la traçabilité des opérations. Enfin, la gestion des accès est assurée via l'entité rôle, permettant de définir les permissions attribuées à chaque utilisateur.
Ainsi, ce MCD reflète à la fois la structure des données et la logique de fonctionnement du système, depuis la collecte des mesures jusqu'à la gestion des incendies.

III. Modèle logique des données

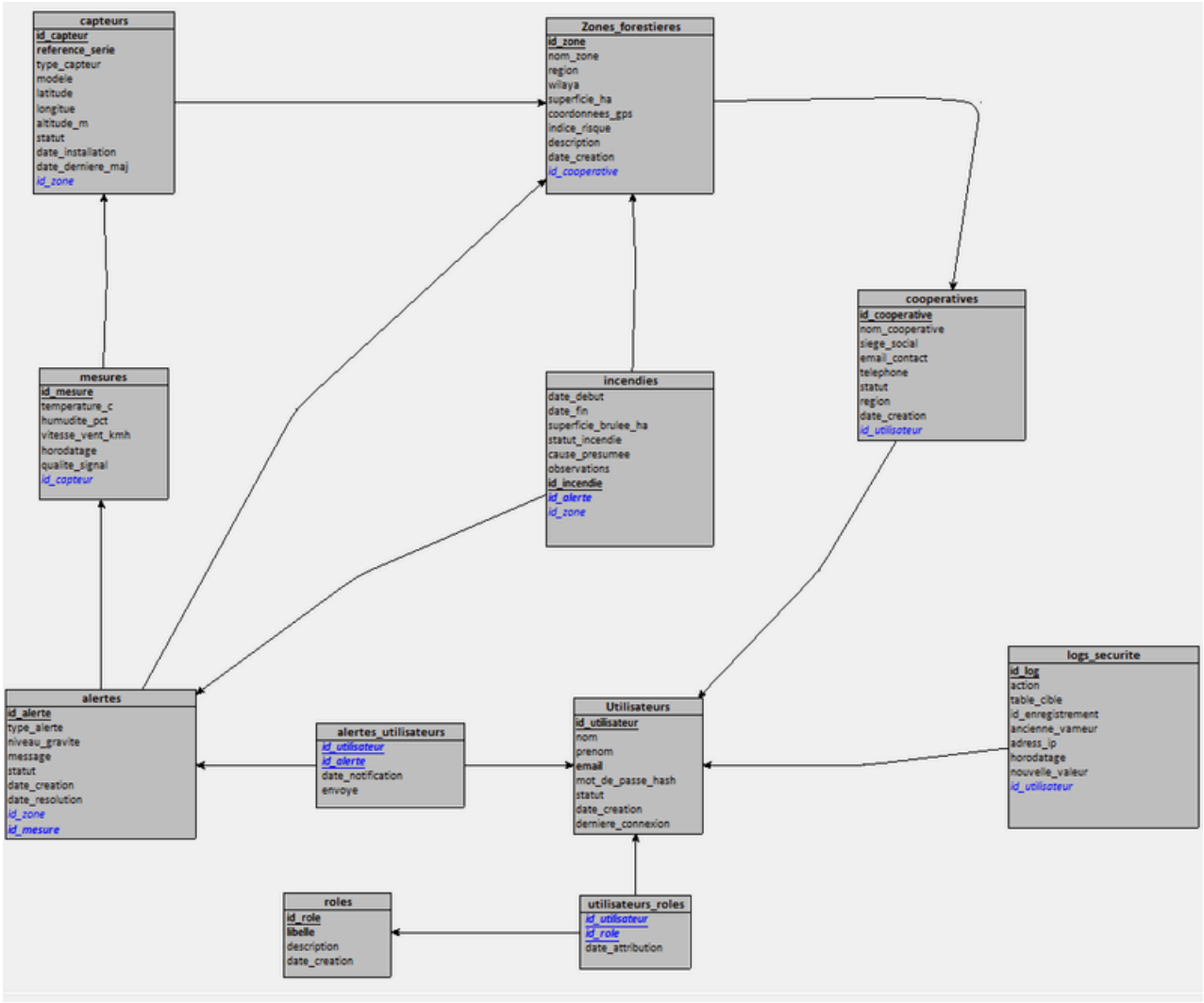

Figure 3 : Modèle logique des données

Le schéma ci-dessus représente le Modèle Logique de Données (MLD), obtenu à partir du MCD précédemment présenté. Il traduit la modélisation conceptuelle en une structure relationnelle exploitable dans un système de gestion de base de données.

Dans ce modèle, chaque entité du MCD est transformée en une table. Par exemple, les entités capteurs, zones forestières, mesures, alertes, incendies, utilisateurs et coopératives deviennent des tables contenant leurs attributs respectifs. Chaque table est identifiée par une clé primaire (id), assurant l'unicité des enregistrements.

Les relations entre les entités sont traduites par des clés étrangères. Ainsi, la relation entre les capteurs et les zones forestières est représentée par la clé étrangère id_zone dans la table capteurs. De même, les tables mesures et alertes sont liées aux capteurs et aux zones via des clés étrangères, assurant la cohérence des données.

Les relations de type plusieurs-à-plusieurs sont modélisées par des tables d'association.

Par exemple, la relation entre utilisateurs et alertes est représentée par la table alertes_utilisateurs, permettant de gérer les notifications envoyées aux utilisateurs.
Ainsi, ce MLD constitue une étape intermédiaire essentielle entre la conception conceptuelle et l'implémentation physique de la base de données, garantissant une structure cohérente, normalisée et adaptée aux besoins du système.

IV. Modèle physique des données

```
/*la structure de la base de données*/
CREATE DATABASE IF NOT EXISTS argan_fire_watch
CHARACTER SET utf8mb4
COLLATE utf8mb4_unicode_ci;

USE argan_fire_watch;

CREATE TABLE roles (
  id_role INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  libelle VARCHAR(50) NOT NULL UNIQUE,
  description TEXT,
  date_creation TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;
CREATE TABLE utilisateurs (
  id_utilisateur INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  nom VARCHAR(100) NOT NULL,
  prenom VARCHAR(100) NOT NULL,
  email VARCHAR(150) NOT NULL UNIQUE,
  mot_de_passe_hash VARCHAR(255) NOT NULL,
    statut  ENUM('ACTIF','INACTIF','SUSPENDU','approved','pending','rejected')
NOT NULL DEFAULT 'ACTIF',
  date_creation TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  derniere_connexion DATETIME
) ENGINE=InnoDB;
CREATE TABLE utilisateurs_roles (
  id_utilisateur INT UNSIGNED NOT NULL,
  id_role INT UNSIGNED NOT NULL,
  date_attribution TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (id_utilisateur, id_role),
   FOREIGN KEY (id_utilisateur) REFERENCES utilisateurs(id_utilisateur) ON
DELETE CASCADE,
  FOREIGN KEY (id_role) REFERENCES roles(id_role) ON DELETE CASCADE
) ENGINE=InnoDB;
```

```
CREATE TABLE zones_forestieres (
  id_zone INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  nom_zone VARCHAR(150) NOT NULL,
  region VARCHAR(100) NOT NULL,
  wilaya VARCHAR(100),
  superficie_ha DECIMAL(10,2) UNSIGNED,
  coordonnees_gps GEOMETRY NOT NULL,
  indice_risque DECIMAL(4,2) UNSIGNED DEFAULT 0.00
    CHECK (indice_risque BETWEEN 0 AND 10),
  description TEXT,
  date_creation TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;

CREATE TABLE capteurs (
  id_capteur INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  reference_serie VARCHAR(100) NOT NULL UNIQUE,
   type_capteur ENUM('TEMPERATURE','MULTI','VENT','HUMIDITE') NOT NULL
DEFAULT 'MULTI',
  modele VARCHAR(100),
  latitude DECIMAL(9,6) NOT NULL,
  longitude DECIMAL(9,6) NOT NULL,
  altitude_m DECIMAL(7,2),
    statut ENUM('ACTIF','INACTIF','EN_MAINTENANCE') NOT NULL DEFAULT
'ACTIF',
  date_installation DATE NOT NULL,
   date_derniere_maj TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
ON UPDATE CURRENT_TIMESTAMP,
  id_zone INT UNSIGNED NOT NULL,
  FOREIGN KEY (id_zone) REFERENCES zones_forestieres(id_zone)
) ENGINE=InnoDB;

CREATE TABLE mesures (
  id_mesure BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  temperature_c DECIMAL(5,2) NOT NULL,
  humidite_pct DECIMAL(5,2) UNSIGNED,
  vitesse_vent_kmh DECIMAL(6,2) UNSIGNED,
  horodatage DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  qualite_signal TINYINT UNSIGNED DEFAULT 100,
  id_capteur INT UNSIGNED NOT NULL,
  FOREIGN KEY (id_capteur) REFERENCES capteurs(id_capteur)
) ENGINE=InnoDB;
```

```
CREATE TABLE alertes (
  id_alerte INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  type_alerte ENUM('AUTOMATIQUE','MANUELLE') NOT NULL DEFAULT 'AUTOMATIQUE',
  niveau_gravite ENUM('INFO','ATTENTION','CRITIQUE') NOT NULL,
  message TEXT NOT NULL,
  statut ENUM('OUVERTE','EN_COURS','RESOLUE','ANNULEE') NOT NULL DEFAULT 'OUVERTE',
  date_creation TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  date_resolution DATETIME,
  id_mesure BIGINT UNSIGNED UNIQUE,
  id_zone INT UNSIGNED NOT NULL,
  FOREIGN KEY (id_mesure) REFERENCES mesures(id_mesure),
  FOREIGN KEY (id_zone) REFERENCES zones_forestieres(id_zone)
) ENGINE=InnoDB;

CREATE TABLE incendies (
  id_incendie INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  date_debut DATETIME NOT NULL,
  date_fin DATETIME,
  superficie_brulee_ha DECIMAL(10,2) UNSIGNED,
  statut_incendie ENUM('EN_COURS','MAITRISE','ETEINT') NOT NULL DEFAULT 'EN_COURS',
  cause_presumee VARCHAR(200),
  observations TEXT,
  id_zone INT UNSIGNED NOT NULL,
  id_alerte INT UNSIGNED UNIQUE,
  FOREIGN KEY (id_zone) REFERENCES zones_forestieres(id_zone),
  FOREIGN KEY (id_alerte) REFERENCES alertes(id_alerte)
) ENGINE=InnoDB;

CREATE TABLE cooperatives (
  id_cooperative INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  nom_cooperative VARCHAR(200) NOT NULL,
  siege_social VARCHAR(300),
  email_contact VARCHAR(150),
  telephone VARCHAR(20),
  statut ENUM('pending', 'approved', 'rejected') NOT NULL DEFAULT 'pending',
  region VARCHAR(100),
  date_creation DATE NOT NULL DEFAULT (CURDATE()),
  id_responsable INT UNSIGNED NOT NULL,
  id_zone INT UNSIGNED,
  FOREIGN KEY (id_responsable) REFERENCES utilisateurs(id_utilisateur),
  FOREIGN KEY (id_zone) REFERENCES zones_forestieres(id_zone) ON DELETE SET NULL
) ENGINE=InnoDB;
```

```sql
CREATE TABLE logs_securite (
  id_log BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  action VARCHAR(50) NOT NULL,
  table_cible VARCHAR(100),
  id_enregistrement VARCHAR(50),
  ancienne_valeur JSON,
  nouvelle_valeur JSON,
  adresse_ip VARCHAR(45),
  horodatage TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  id_utilisateur INT UNSIGNED,
  FOREIGN KEY (id_utilisateur) REFERENCES utilisateurs(id_utilisateur)
) ENGINE=InnoDB;

CREATE TABLE alertes_utilisateurs (
  id_alerte INT UNSIGNED,
  id_utilisateur INT UNSIGNED,
  date_notification DATETIME DEFAULT CURRENT_TIMESTAMP,

  PRIMARY KEY (id_alerte, id_utilisateur),

  FOREIGN KEY (id_alerte) REFERENCES alertes(id_alerte),
  FOREIGN KEY (id_utilisateur) REFERENCES utilisateurs(id_utilisateur)
);
```

V. Diagramme des cas d'utilisation

1-Diagramme des cas d'utilisation pour l'administrateur

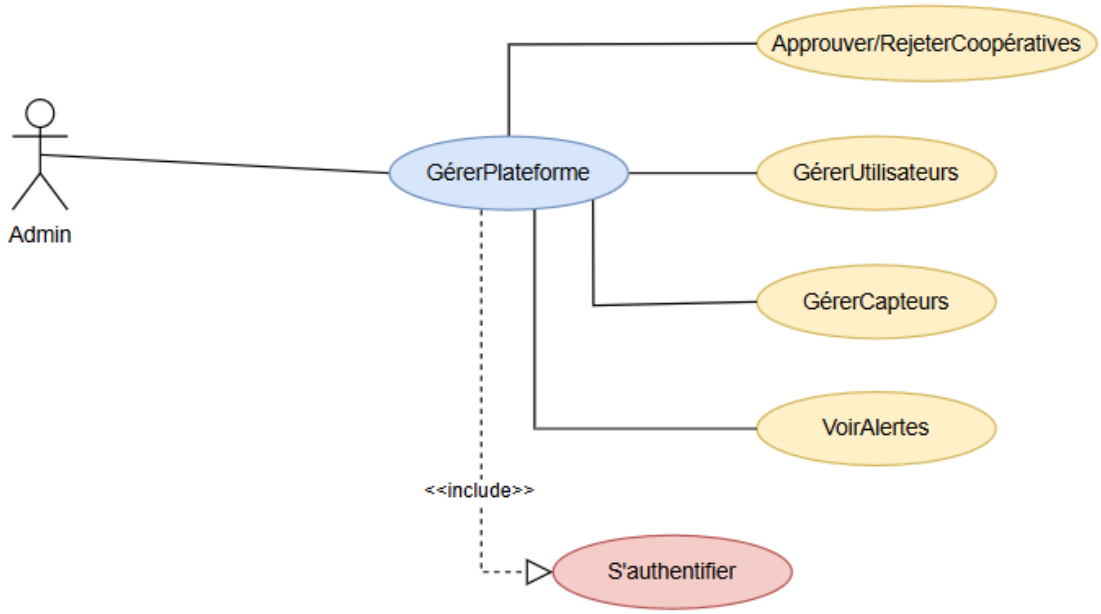

Figure 4 : Diagramme des cas d'utilisations pour l'administrateur

Le diagramme ci-dessus représente les différents cas d'utilisation de l'administrateur dans l'application. L'administrateur est l'acteur principal disposant des droits complets pour gérer la plateforme.
Avant d'accéder aux différentes fonctionnalités du système, l'administrateur doit obligatoirement s'authentifier. Cette étape est essentielle pour garantir la sécurité et le contrôle d'accès à l'application.
L'action principale « Gérer Plateforme » regroupe plusieurs fonctionnalités essentielles :

- la gestion des utilisateurs,
- la gestion des capteurs,
- la consultation des alertes,
- l'approbation ou le rejet des coopératives.

Ce diagramme permet ainsi de mieux comprendre les responsabilités de l'administrateur ainsi que les différentes actions qu'il peut effectuer pour assurer le bon fonctionnement de la plateforme.

2-Diagramme des cas d'utilisation pour un pompier

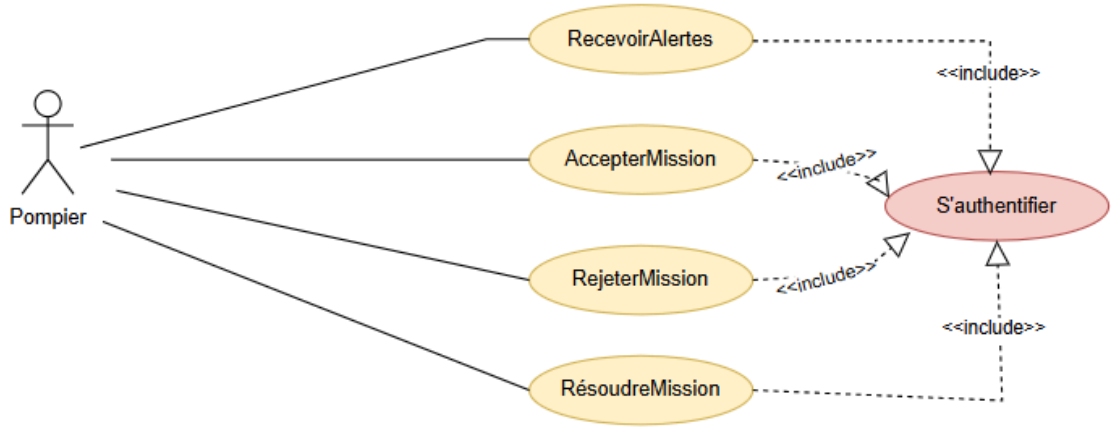

Figure 5 : Diagramme des cas d'utilisations pour le pompier

Le diagramme ci-dessus représente les cas d'utilisation de l'acteur pompier dans l'application. Le pompier intervient principalement pour gérer les alertes liées aux incendies.
Avant d'accéder aux fonctionnalités du système, il doit obligatoirement s'authentifier afin de sécuriser l'accès.
Les principales actions du pompier sont :

- recevoir les alertes générées par le système,
- accepter ou rejeter une mission selon la situation,
- et résoudre la mission après intervention sur le terrain.

Ce diagramme met ainsi en évidence le rôle opérationnel du pompier dans le système, depuis la réception des alertes jusqu'à la gestion des interventions.

3-Diagramme des cas d'utilisation pour un responsable d'une coopérative

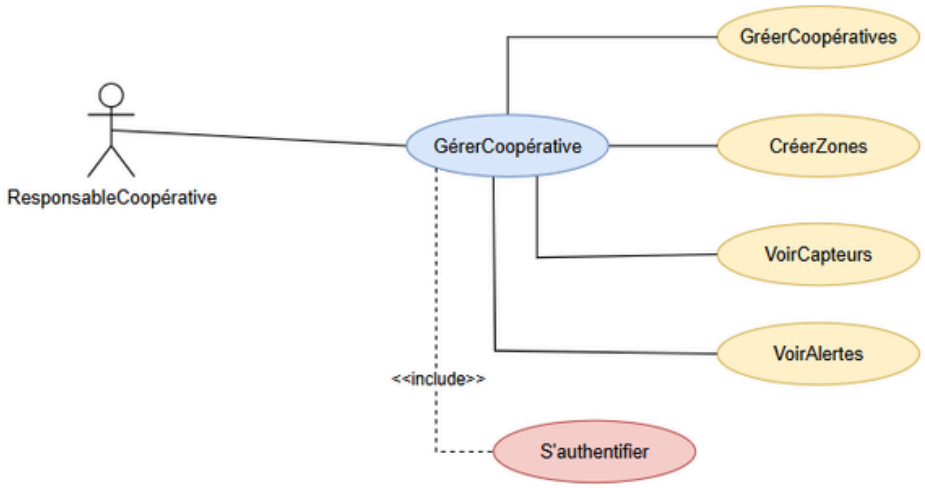

Figure 6 : Diagramme des cas d'utilisations pour le responsable de la coopérative

Le diagramme ci-dessus représente les cas d'utilisation de l'acteur responsable de coopérative dans l'application. Cet acteur est chargé de gérer les activités liées à sa coopérative.
Avant d'accéder aux différentes fonctionnalités, il doit obligatoirement s'authentifier afin de sécuriser l'accès au système.
L'action principale « Gérer Coopérative » regroupe plusieurs fonctionnalités :

- la création de coopératives,
- la gestion des zones forestières,
- la consultation des capteurs,
- ainsi que la visualisation des alertes.

Ce diagramme met en évidence le rôle du responsable de coopérative dans le suivi et la gestion des ressources, ainsi que sa contribution à la surveillance des zones forestières.

VI. Script SQL : Triggers et procédures

1-Procédures

Procédure de calcul de propagation :

```
CREATE PROCEDURE sp_calcul_propagation(
 IN p_temperature DECIMAL(5,2),
 IN p_humidite DECIMAL(5,2),
 IN p_vent DECIMAL(6,2),
 OUT p_prob INT
)
BEGIN
 DECLARE v_prob INT DEFAULT 0;
 -- Température (50%)
 SET v_prob = v_prob + LEAST(p_temperature * 0.5, 50);
 -- Vent (30%)
 SET v_prob = v_prob + LEAST(p_vent * 0.3, 30);
 -- Humidité (inverse)
 SET v_prob = v_prob + LEAST((100 - p_humidite) * 0.2, 20);
 SET p_prob = ROUND(v_prob);
END$$
```

Procédure d'ajout d'une mesure :

```
CREATE PROCEDURE sp_ajouter_mesure(
 IN p_id_capteur INT,
 IN p_temperature DECIMAL(5,2),
 IN p_humidite DECIMAL(5,2),
 IN p_vent DECIMAL(6,2)
)
BEGIN
INSERT INTO mesures(
 id_capteur,
 temperature_c,
 humidite_pct,
 vitesse_vent_kmh
)
VALUES(
 p_id_capteur,
 p_temperature,
 p_humidite,
 p_vent
);
END$$
```

2-Triggers

Trigger de Détection Automatique d'Incendie et Calcul de Propagation

```
CREATE TRIGGER trg_detection_incendie
AFTER INSERT ON mesures
FOR EACH ROW
BEGIN
  DECLARE v_id_zone INT UNSIGNED;
  DECLARE v_prob INT;
  -- zone
  SELECT id_zone INTO v_id_zone
  FROM capteurs
  WHERE id_capteur = NEW.id_capteur;
  --     calcul propagation
  CALL sp_calcul_propagation(
    NEW.temperature_c,
    IFNULL(NEW.humidite_pct,50),
    IFNULL(NEW.vitesse_vent_kmh,0),
    v_prob
  );
  --     condition incendie
  IF NEW.temperature_c > 50.00 THEN
INSERT INTO alertes (
     type_alerte,
     niveau_gravite,
     message,
     statut,
     id_mesure,
     id_zone,
     probabilite_propagation
   ) VALUES (
     'AUTOMATIQUE',
     'CRITIQUE',
     CONCAT(
       '    ALERTE INCENDIE - Temp: ', NEW.temperature_c,
       '°C | Propagation: ', v_prob, '%'
     ),
     'OUVERTE',
     NEW.id_mesure,
     v_id_zone,
     v_prob
   );
  END IF;
END$$
```

Trigger pour la création de la notification :

```
CREATE TRIGGER trg_notification_pompiers
AFTER INSERT ON alertes
FOR EACH ROW
BEGIN
  -- Notifier tous les pompiers actifs si alerte CRITIQUE
  IF NEW.niveau_gravite = 'CRITIQUE' THEN
    INSERT INTO alertes_utilisateurs (id_alerte, id_utilisateur)
    SELECT NEW.id_alerte, u.id_utilisateur
    FROM utilisateurs u
    INNER JOIN utilisateurs_roles ur
      ON u.id_utilisateur = ur.id_utilisateur
    INNER JOIN roles r
      ON ur.id_role = r.id_role
    WHERE r.libelle = 'POMPIER'
    AND u.statut = 'ACTIF';
  END IF;
END$$
```

Trigger pour le clacul de l'indice du risque :

```
CREATE TRIGGER trg_recalcul_indice_risque
AFTER INSERT ON mesures
FOR EACH ROW
BEGIN
  DECLARE v_id_zone INT UNSIGNED;
  DECLARE v_indice DECIMAL(4,2);
  DECLARE v_temp_norm DECIMAL(5,4);
  DECLARE v_hum_norm DECIMAL(5,4);
  DECLARE v_vent_norm DECIMAL(5,4);
  SELECT id_zone INTO v_id_zone
  FROM capteurs
  WHERE id_capteur = NEW.id_capteur;
  -- Normalisation des valeurs
  SET v_temp_norm = LEAST(NEW.temperature_c / 60.0, 1.0);
  SET v_hum_norm = 1.0 - LEAST(IFNULL(NEW.humidite_pct,50) / 100.0, 1.0);
  SET v_vent_norm = LEAST(IFNULL(NEW.vitesse_vent_kmh,0) / 100.0, 1.0);
  -- Calcul pondéré
  SET v_indice = ROUND(
    (v_temp_norm * 0.50 +
     v_hum_norm * 0.30 +
     v_vent_norm * 0.20) * 10.0, 2);
  UPDATE zones_forestieres
  SET indice_risque = v_indice
  WHERE id_zone = v_id_zone;
END$$
```

Prompts utilisés :

Génération du MCD, MLD et MPD :

Agis comme un expert en conception de bases de données et en méthode MERISE. Je travaille sur un projet universitaire appelé Souss-Massa Resilience Prototype 2026. Sujet de mon équipe : Argan-Fire Watch Objectif du système : Créer un système d'information qui permet de détecter les incendies dans les forêts d'arganiers au Maroc grâce à des capteurs qui mesurent :

- la température
- l'humidité
- la vitesse du vent Lorsque la température dépasse 50°C, le système doit générer automatiquement une alerte incendie pour prévenir les pompiers. Le système doit aussi permettre :
- le suivi des capteurs installés dans les zones forestières
- l'enregistrement des mesures envoyées par les capteurs
- la détection des incendies
- la gestion des alertes
- la visualisation des zones à risque
- la gestion des coopératives d'argan Acteurs du système :

1. Admin
   - gère la plateforme
   - gère les capteurs
   - gère les utilisateurs
2. Pompier
   - reçoit les alertes incendie
   - consulte les zones en feu
3. Utilisateur coopérative
   - peut créer et gérer une coopérative d'argan
   - peut consulter les informations sur les incendies
4. Visiteur simple
   - peut consulter certaines informations publiques
   - n'a pas de compte
   - ses informations ne sont pas stockées dans la base de données Contraintes techniques du projet :

- La base de données doit être relationnelle (MySQL ou PostgreSQL).
- Le modèle doit respecter la normalisation jusqu'à la 3FN.
- Le système doit contenir des triggers et des procédures stockées.
- Le système doit inclure un RBAC (Role-Based Access Control) pour gérer les rôles des utilisateurs.
- Le modèle doit être cohérent avec la méthode MERISE. Tâches que je veux que tu réalises :

5. Analyse le système et identifie les entités principales.
6. Identifie leurs attributs.

7. Identifie les relations entre les entités.
8. Donne les règles de gestion principales. Ensuite :
9. Génère le MCD (Modèle Conceptuel de Données) MERISE complet :
    - entités
    - attributs
    - associations
    - cardinalités
10. Transforme ce modèle en MLD (Modèle Logique de Données) :
    - tables relationnelles
    - clés primaires
    - clés étrangères
11. Génère ensuite le MPD SQL pour MySQL avec les tables.
12. Vérifie que le modèle respecte bien la 3ème forme normale (3FN).
13. Propose aussi quelques triggers possibles pour ce système, par exemple :
    - déclencher une alerte si température > 50°C
    - enregistrer les logs de sécurité
    - recalculer l'indice de risque d'incendie. Structure ta réponse de manière claire et pédagogique.

VII. Conclusion

Ce chapitre a présenté les différentes étapes de conception du système, à travers le MCD, le MLD et le MPD, ainsi que leur implémentation via les scripts SQL. Les diagrammes de cas d'utilisation ont permis de définir les interactions entre les acteurs et le système. Enfin, l'utilisation des prompts IA a contribué à faciliter et accélérer le développement.

CHAPITRE 3 : OUTILS,TECHNOLOGIES ET PROMPTS UTILISÉS

I. INTRODUCTION

Dans ce chapitre, nous présentons les fondements techniques et organisationnels de notre projet Argan Fire Watch, une plateforme web dédiée à la surveillance et à l'alerte précoce des incendies dans les forêts d'arganier de la région Souss-Massa.

Nous commencerons par décrire l'objectif global de notre projet ainsi que le public cible, à savoir les trois catégories d'acteurs principales : les coopératives locales, les pompiers et chefs d'équipe, ainsi que les administrateurs. Nous mettrons en lumière les besoins auxquels cette plateforme répond et les bénéfices qu'elle apporte en termes de réduction des délais de détection et d'amélioration de la coordination des interventions.

Ensuite, nous détaillerons l'ensemble des technologies utilisées pour la réalisation de ce projet. Nous présenterons successivement les langages et frameworks employés, à savoir Python avec le framework Flask pour le backend, React pour le frontend, ainsi que MySQL comme système de gestion de base de données. Enfin, nous évoquerons les outils de développement (VS Code, Looping) et les outils d'intelligence artificielle (ChatGPT, Grok, Claude, DeepSeek) qui ont facilité et enrichi notre travail tout au long du cycle de conception et de développement.

II. Technologies Utilisées

1-Langages et Frameworks Utilisés

a-Python

Figure 7 : Logo Python

Python est un langage de programmation simple, lisible et très puissant. Il a été choisi pour développer la partie backend du projet grâce à sa grande simplicité et à ses nombreuses bibliothèques

b-Flask

Figure 8 : Logo Flask

Flask est un framework web léger écrit en Python. Il a été utilisé pour créer l'API backend, gérer les routes, l'authentification des utilisateurs et la logique du système.

c-React

Figure 9 : Logo React

React est une bibliothèque JavaScript permettant de créer des interfaces utilisateur modernes et interactives. Elle a été utilisée pour développer le frontend de la plateforme.

d-MySQL

Figure 10 : Logo MySQL

MySQL est un système de gestion de base de données relationnelle open-source. Il a été utilisé pour stocker toutes les informations du projet (utilisateurs, coopératives, zones forestières, capteurs, alertes et mesures). Il permet également de gérer les données spatiales des zones.

2-Outils de Développement

a-VS Code

Figure 11 : Logo VS Code

b-Antigravity :

Figure 12 : Logo Antigravity

c-Looping

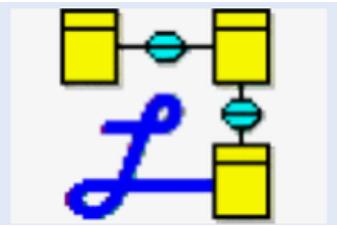

Figure 13 : Logo Looping

3-Outils d'Intelligence Artificielle

ChatGPT

Figure 14:Logo ChatGPT

Grok

Figure 15 : Logo Grok

Claude

Figure 16 : Logo Claude

DeepSeek

Figure 17 : Logo DeepSeek

III. Prompt Utilisées :

1. Gestion des coopératives en attente :

Crée une route Flask GET /cooperatives/pending qui récupère les coopératives avec statut "pending" en joignant la table utilisateurs pour afficher les informations du responsable.

2. Validation d'une coopérative

Génère une route permettant d'approuver une coopérative et de mettre à jour automatiquement le statut de l'utilisateur responsable.

3. Gestion des utilisateurs

Crée une route Flask GET /users qui retourne la liste des utilisateurs avec leurs rôles et leurs informations associées.

4. Structure de la base de données et affichage des demandes en attente :

Here is the structure of my cooperatives and users tables. How can I display pending requests in an admin interface?

5. **Absence de données dans la page des demandes en attente**

Why are no records displayed on the pending requests page even though data exists in the database?

6. **Débogage de l'échec d'inscription**

Why does user registration fail even though the form is correctly filled on the frontend?

7. **Cohérence des données entre React et Flask :**

How can I ensure consistency between data sent from React (JSON) and data received in Flask?

8. **Structuration des données capteurs pour le frontend**

What is the best way to structure sensor data (temperature, humidity, wind) for frontend consumption?

9. **Implémentation de la logique de propagation des incendies**

Je souhaite implémenter une logique de calcul de la probabilité de propagation en fonction du vent et de l'humidité.

10. **Mise en place d'un système de notification pour les pompiers**

How can I ensure consistency between data sent from React (JSON) and data received in Flask?

Ici,nous donnons nos idées à un LLM comme ChatGPT ou Claude. Ensuite, il nous génère un prompt plus détaillé, puis nous le donnons au chat d'Antigravity pour générer le code.

11. **Formulaire d'inscription pour une coopérative**

Build a multi-step Cooperative Registration Form in React for a platform called Argan-Fire Watch.
Tech: React, Tailwind CSS, Axios, Leaflet.js (react-leaflet). No backend calls needed — just the frontend form with a final submit to POST /auth/register.
4 steps with a progress bar at the top showing current step:
Step 1 — Personal Information

- First name (prenom)
- Last name (nom)
- Email address
- Password (minimum 8 characters)
- Confirm password (validate they match)

Step 2 — Cooperative Information

- Cooperative name
- Region (dropdown: Agadir-Ida-Outanane, Taroudant, Tiznit, Chtouka-Ait-Baha, Inezgane-Ait-Melloul)
- Address
- Phone number

Step 3 — Zone Selection on Map

- Full screen Leaflet map centred on Souss-Massa region Morocco (coordinates 30.4278, -9.5981, zoom 9)
- The cooperative owner draws a polygon on the map to select the area of his cooperative zone using Leaflet Draw plugin
- Below the map show the selected area in hectares (calculated from the polygon)
- Zone name input field below the map
- If no polygon drawn, show a warning "Please select your zone on the map"

Step 4 — Review & Submit

- Show a summary of all data entered in steps 1, 2 and 3 in a clean read-only card
- Show the selected zone polygon on a small Leaflet map (read-only, no drawing)
- Checkbox: "I confirm all information is correct"
- Submit button — on click call POST /auth/register with all data as JSON including the polygon coordinates
- While submitting show loading spinner on button
- On success hide the form and show a success card: "Your registration has been submitted. The admin will review your account within 24 hours."
- On error show red error message at top

Navigation:

- Next button to go to next step (validates current step fields before proceeding)
- Back button to go to previous step
- Step indicator at top: 4 dots or numbered steps showing which step is active, completed, or pending

Style: Professional and clean. Green and orange accent colors. Dark theme. Matches a fire monitoring platform. Fully responsive on mobile. Smooth transition animation between steps.

12. Connexion du formulaire au backend :

I have a multi-step Cooperative Registration Form in React (4 steps with Leaflet map zone selection) for a platform called Argan-Fire Watch. I need to connect it to a Flask backend.
Current frontend:

- Step 1: personal info (nom, prenom, email, password)
- Step 2: cooperative info (name, region, address, phone)
- Step 3: zone selection on Leaflet map (polygon coordinates + zone name + surface in hectares)
- Step 4: review and submit

What I need:
1. Flask route to add — POST /auth/register:

- Receives JSON with all form data
- Creates a new user in user_1 table with role = cooperative and a bcrypt hashed password
- Creates a new cooperative in cooperative_1 table linked to the user with status = pending
- Creates a new zone in zone_argane_1 table linked to the cooperative, storing the polygon coordinates as JSON in a coordinates column
- Returns { "message": "Registration submitted" } on success
- Returns { "error": "Email already exists" } if email is taken

2. Database changes needed:

- Add status column to cooperative_1 table: ENUM('pending', 'approved', 'rejected') DEFAULT 'pending'
- Add coordinates column to zone_argane_1 table: TEXT to store polygon GeoJSON

3. Frontend changes needed:

- In the final submit button, call POST /auth/register with all collected data as JSON
- Handle success: show success card
- Handle error: show error message from API response
- Include CORS headers — Flask backend runs on http://localhost:5000

My current tech stack:

- Frontend: React + Tailwind + Axios + react-leaflet
- Backend: Flask + mysql-connector-python + flask-jwt-extended + flask-cors
- Database: MySQL via XAMPP phpMyAdmin

Give me:

1. The SQL to update the database (ALTER TABLE statements)
2. The complete Flask route POST /auth/register in app/routes/auth_routes.py
3. The updated submit function in the React form that calls the API correctly

13. Création du tableau de bord des pompiers :

Redesign the firefighter dashboard to match the style used for firefighters. Remove all static data and translate everything to English. Also add a Map page identical to the one in the admin dashboard (copy it exactly).
Add charts to the dashboard as well.
The Alerts page should also be redesigned to match the one in the admin dashboard.
When a new alert is generated, display a popup with a sound notification. The popup should show the alert location and the cooperative name, and include two buttons: “Take Mission” and “Ignore”.
This popup should only appear if the firefighter’s status is “Available”.
When the firefighter takes the mission, add a button that allows them to change the alert status to “Resolved”.
After that, display a form containing the fields from the incendie table. The firefighter must fill in this form, and the submitted data should be saved in the incidents table.

14. Ajout de la vue détaillée d'une coopérative dans les validations en attente

Go to the admin dashboard, in Pending Approvals add a 'View Details' button on each cooperative card. The modal should display all cooperative information including area and zones before approving or rejecting it. Add a confirmation popup after approve/reject."
Erreurs :

- Au début la modal ne montrait que du texte, sans la zone dessinée.

15. Ajout d'une heatmap de température dans les zones :

Implement a temperature heatmap in the System Map using Leaflet heatmap. The heatmap should use sensor temperatures and visually show hot and cold areas.

16. Ajout d'une carte interactive dans la page des zones

Go to the Cooperative Dashboard, in the 'My Zones' page add an 'Expand View' button on each zone card. When the user clicks it, open a modal containing a large interactive map focused on the selected zone. The selected zone must appear highlighted with a strong border and colored fill. All the other zones of the cooperative should also appear in the background with lighter colors for context. The modal should also display the zone name, area, coordinates and number of sensors."
Erreurs :

- Au début seule la zone sélectionnée apparaissait, les autres zones n'étaient pas visibles.
- Les autres zones étaient affichées avec la même couleur que la zone principale, ce qui rendait la lecture difficile.
- La modal ne montrait pas les informations détaillées de la zone.

Problèmes rencontrés avec l'IA

L'un des principaux problèmes rencontrés lors de l'utilisation de l'intelligence artificielle réside dans le fait qu'elle proposait parfois des modifications non souhaitées, notamment des changements au niveau du style ou de la structure du code, alors que le besoin était limité à des corrections précises. Cela a entraîné une perte de temps, car il a fallu réadapter le code afin de préserver l'interface et l'architecture initiales. De plus, certaines réponses incluaient des refactorisations complètes qui ne correspondaient pas toujours aux attentes du projet.

Par ailleurs, certaines propositions étaient trop générales et ne prenaient pas suffisamment en compte le contexte spécifique, ce qui nécessitait des reformulations ou des précisions supplémentaires. L'IA ne détectait pas non plus systématiquement les erreurs liées à l'environnement, telles que les conflits de ports entre React et Flask ou les mauvaises configurations des routes, ce qui a ralenti le processus de débogage.

En outre, les solutions proposées étaient parfois théoriques et demandaient une adaptation avant d'être directement exploitables dans le code réel. Dans certains cas, l'IA ne prenait pas correctement en compte les relations complexes entre les tables de la base de données, notamment celles impliquant les rôles utilisateurs, ce qui a entraîné des incompréhensions lors de l'implémentation de certaines fonctionnalités comme le système d'authentification.

De manière générale, le diagnostic des erreurs nécessitait plusieurs interactions successives, une seule réponse n'étant pas toujours suffisante pour résoudre entièrement un problème.

Enfin, certaines limites ont été observées dans la compréhension globale de l'architecture, notamment en ce qui concerne les triggers SQL, les procédures stockées ou encore la gestion des valeurs NULL, ainsi que dans l'interprétation des relations entre les entités (capteurs, zones et coopératives).
Ces éléments montrent que, malgré son utilité, l'intelligence artificielle reste un outil d'assistance qui nécessite une validation, une adaptation et un esprit critique de la part du développeur.

Aspects appréciés

Malgré les difficultés rencontrées, l'utilisation de l'intelligence artificielle s'est révélée globalement bénéfique dans le cadre de ce projet. Elle a permis d'identifier rapidement certaines erreurs techniques, telles que le mauvais placement des hooks ou l'utilisation de données statiques au lieu de données dynamiques. L'IA a également fourni des solutions fonctionnelles accompagnées d'explications claires, ce qui a facilité la compréhension des problèmes et accéléré le processus de développement.
Par ailleurs, elle a permis d'obtenir rapidement des éclaircissements sur des aspects techniques complexes, notamment la configuration des routes Flask, l'écriture des requêtes SQL et la communication entre le frontend et le backend. Cette assistance a contribué à une meilleure compréhension des interactions entre les différentes couches de l'application, à savoir React, l'API Flask et la base de données.
Les exemples de code proposés ont facilité l'implémentation rapide de plusieurs fonctionnalités, telles que l'inscription, l'authentification et la récupération des données pour le tableau de bord. L'IA a également aidé à structurer le backend en suggérant des endpoints cohérents pour la gestion des capteurs et des données environnementales.
En outre, elle a permis de gagner un temps considérable lors des phases de débogage, en identifiant les causes probables des erreurs et en proposant des corrections ciblées. Son approche progressive et interactive a également joué un rôle pédagogique important, en accompagnant étape par étape la résolution des problèmes rencontrés.
Enfin, l'intelligence artificielle a apporté une réelle valeur ajoutée au projet en suggérant des améliorations pertinentes, telles que l'intégration d'une logique de calcul de la propagation des incendies.

Ainsi, elle s'est révélée être un outil efficace d'assistance au développement, à condition d'être utilisée de manière critique, avec des prompts précis et une validation constante des solutions proposées.

IV. Conclusion

En conclusion, ce chapitre a permis de présenter l'ensemble des choix technologiques retenus pour la conception et le développement de la plateforme Argan Fire Watch. L'architecture technique repose sur une combinaison cohérente et performante : Python avec le framework Flask pour un backend léger et robuste, React pour une interface utilisateur moderne et interactive, ainsi que MySQL pour une gestion fiable des données, y compris les données spatiales.

Par ailleurs, les outils de développement tels que VS Code et Looping, ainsi que le recours aux outils d'intelligence artificielle (ChatGPT, Grok, Claude, DeepSeek), ont contribué à faciliter et à optimiser le travail tout au long du cycle de réalisation. Dans une logique d'analyse critique de l'usage de l'IA, ce chapitre a également mis en évidence les prompts utilisés, les problèmes rencontrés lors de l'interaction avec ces outils, ainsi que les aspects appréciés, permettant ainsi de mettre en évidence ses points forts et ses limites dans un contexte réel de développement logiciel.

CONCLUSION GENERALE

La plateforme Argan Fire Watch développée représente un outil moderne, pratique et indispensable pour les coopératives, les pompiers et les administrateurs souhaitant organiser efficacement la surveillance et la lutte contre les incendies au sein des forêts d'arganier de la région Souss-Massa. Grâce à ses fonctionnalités intuitives et à son interface conviviale, elle permet de surveiller en temps réel les zones forestières, détecter précocement les départs de feu, recevoir des alertes instantanées sur WhatsApp et coordonner les interventions en toute simplicité.

L'intégration de cartes interactives, de capteurs IoT mesurant la température, l'humidité et la vitesse du vent, ainsi que d'un système de notifications en temps réel via WebSocket, garantit une expérience utilisateur optimale. Le choix du framework Flask (Python) pour le backend, associé à React pour le frontend et à MySQL pour la gestion des données spatiales, assure à la plateforme une compatibilité fluide et une réactivité exemplaire.

En somme, cette plateforme répond parfaitement aux besoins de protection des forêts d'arganier en offrant aux différents acteurs un outil fiable, intuitif et performant pour mieux anticiper et gérer les risques d'incendies. Elle constitue un véritable compagnon numérique qui préserve un écosystème unique classé par l'UNESCO, réduit les délais de détection des feux, limite les pertes environnementales et humaines, et encourage une gestion intelligente et responsable des ressources forestières.